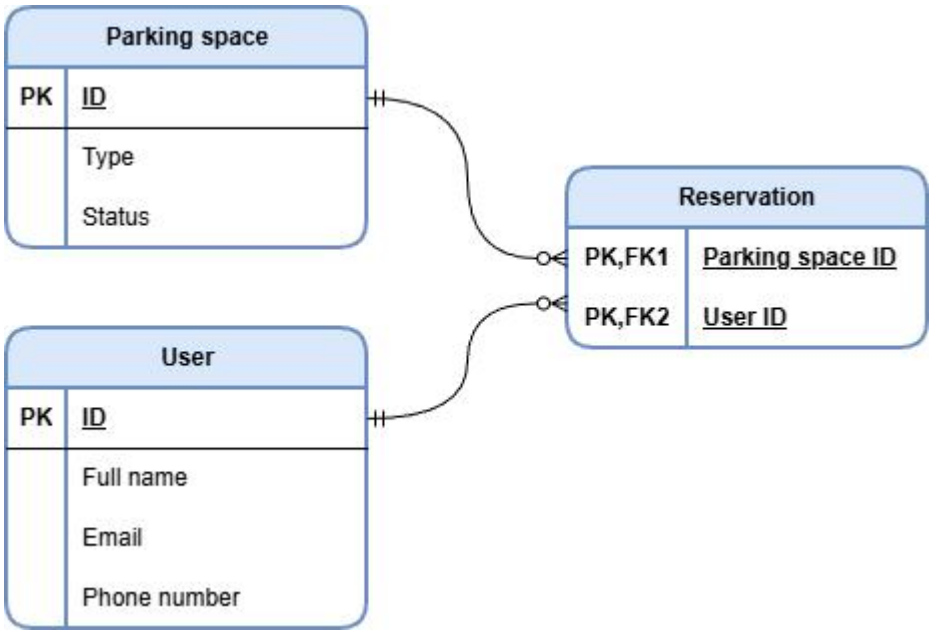
Практична робота №4. Перетворення ER-діаграми у реляційну схему

Ісай Дмитро



Таблиці, атрибути, домени та первинні ключі

Таблиця: User

- Атрибути та домени:
  - ID — INT (або BIGINT), Первинний ключ (PK), Auto Increment.
  - Full name — VARCHAR(100).
  - Email — VARCHAR(255).
  - Phone number — VARCHAR(20).
- Первинний ключ (PK): ID

Таблиця: Parking space

- Атрибути та домени:
  - ID — INT (або BIGINT), Первинний ключ (PK), Auto Increment.
  - Type — VARCHAR(50) (наприклад: Standard, Electric, Disabled, VIP).
  - Status — VARCHAR(20) (наприклад: Available, Occupied, Reserved, Maintenance).
- Первинний ключ (PK): ID

Таблиця: Reservation

- Атрибути та домени:
  - Parking space ID — INT (Зовнішній ключ FK1, частина складеного PK).
  - User ID — INT (Зовнішній ключ FK2, частина складеного PK).
- Первинний ключ (PK): Складений ключ (Parking space ID, User ID).

Таблиця відповідності: «сутність → таблиця, зв'язок → правило перетворення»

Сутність / Зв'язок у концептуальній моделі	Об'єкт реляційної схеми	Правило перетворення
Сутність User	Таблиця User	Пряме перетворення сутності в таблицю бази даних.
Сутність Parking space	Таблиця Parking space	Пряме перетворення сутності в таблицю бази даних.
Зв'язок M:N між User та Parking space	Проміжна таблиця Reservation	Зв'язок **багато-до-багатьох (M:N)** розбивається на два зв'язки **1:N** за допомогою створення окремої сполучної таблиці зі складеним первинним ключем (Parking space ID, User ID).

Обмеження цілісності для кожної таблиці

Таблиця User

- Первинний ключ: ID (NOT NULL, UNIQUE).
- Унікальність (UNIQUE): Email (пошта повинна бути унікальною для кожного користувача).
- Обов'язковість (NOT NULL): Full name, Email, Phone number.
- Перевірка діапазонів/формату (CHECK): Email має відповідати шаблону формату пошти (%@%.%).

Таблиця Parking space

- Первинний ключ: ID (NOT NULL, UNIQUE).

- Обов'язковість (NOT NULL): Type, Status.
- Значення за замовчуванням (DEFAULT): Status = 'Available'.
- Обмеження значень (CHECK):
  - Status IN ('Available', 'Occupied', 'Reserved', 'Maintenance')
  - Type IN ('Standard', 'Electric', 'Disabled', 'VIP')

Таблиця Reservation

- Первинний ключ: Складений PK (Parking space ID, User ID).
- Зовнішній ключ Parking space ID (FK1):
  - Посилається на Parking space(ID).
  - Обов'язковість: NOT NULL.
  - Поведінка при видаленні/оновленні: ON DELETE CASCADE / ON UPDATE CASCADE (якщо видаляється паркомісце, бронювання також видаляється).
- Зовнішній ключ User ID (FK2):
  - Посилається на User(ID).
  - Обов'язковість: NOT NULL.
  - Поведінка при видаленні/оновленні: ON DELETE CASCADE / ON UPDATE CASCADE (якщо видаляється користувач, його бронювання видаляються).

Пояснення: Чому зв'язки M:N перетворюються на окремі таблиці

У реляційній теорії та СУБД реляційна модель не підтримує пряму реалізацію зв'язків M:N (багато-до-багатьох) з наступних причин:

- Атомарність даних (Перша нормальна форма - 1NF): Якби ми намагалися зберегти зв'язок M:N в одній із таблиць (наприклад, у таблиці User), нам довелося б зберігати масив або список ID паркомісць в одного користувача (наприклад, 1, 4, 7), що порушує вимогу атомарності значень в осередку.

- Дублювання даних: Якби ми створювали окремий рядок для кожного бронювання у таблиці User, нам довелося б дублювати персональні дані користувача (Full name, Email) для кожного нового паркомісця.

- Забезпечення посилальної цілісності: Реляційні бази даних не можуть автоматично перевіряти цілісність та коректність для списків чи масивів ключів у межах одного поля.

Створення окремої сполучної таблиці (такої як Reservation) перетворює складний зв'язок M:N на два прості зв'язки 1:N (Parking

space -> Reservation та User -> Reservation), що забезпечує нормалізацію БД і підтримку посилальної цілісності СУБД.

Відповіді на питання для захисту

1. Чому для зв'язку 1:N зовнішній ключ розміщують на стороні «багато»?

Відповідь: На стороні «багато» кожен запис відповідає лише одному запису зі сторони «один». Тому в таблиці на стороні «багато» достатньо додати один додатковий стовпець для збереження ID батьківського запису (зовнішній ключ). Якби ми спробували розмістити зовнішній ключ на стороні «один», нам довелося б зберігати множинні значення (список ID) в одному полі, що порушує 1NF.

2. Що таке посилальна цілісність і як її забезпечити?

Відповідь: Посилальна цілісність (Referential Integrity) — це правило цілісності БД, яке гарантує, що значення зовнішнього ключа (FK) в дочірній таблиці завжди відповідає існуючому значенню первинного ключа (PK) у батьківській таблиці (або є NULL, якщо це дозволено). Вона забезпечується за допомогою:

- Створення обмежень FOREIGN KEY у DDL-скриптах БД.
- Правил обробки видалення/зміни батьківських записів (CASCADE, RESTRICT/NO ACTION, SET NULL).

3. Як перетворити зв'язок M:N без втрати даних?

Відповідь: Шляхом створення додаткової (проміжної/асоціативної) таблиці. Ця таблиця містить як мінімум два зовнішні ключі, які вказують на первинні ключі двох вихідних таблиць. Зазвичай комбінація цих двох FK утворює складений первинний ключ проміжної таблиці, що гарантує збереження всіх відносин між об'єктами без дублювання базових атрибутів.

4. Що станеться, якщо видалити запис батьківської таблиці без каскаду?

Відповідь: Поведінка залежить від налаштованого правила зовнішнього ключа:

- RESTRICT / NO ACTION (за замовчуванням): СУБД заблокує операцію видалення та поверне помилку цілісності (Foreign Key Constraint Violation), оскільки є дочірні записи, які посилаються на цей запис.

- SET NULL: Батьківський запис буде видалено, а у відповідних зовнішніх ключах дочірньої таблиці значення заміняться на NULL (якщо поле FK не має обмеження NOT NULL).